IT Shell & Kernel Programming
Understanding the Shell

A. Mustafi

January 30, 2023

The Linux shell is an interpreter of commands. Any time we see the Linus prompt on the screen the Shell is waiting to receive a new command. Interestingly the shell is just another running process which allows other processes to execute.

The shell can not only receive commands but can also parse wild cards before passing the command onto the kernel for execution. Thus one of the key tasks for the shell is Pattern Matching in the form of wildcards.

0.1 Common Wildcards

The shell wild cards are heavily inspired by the pattern matching system available in PERL. Some of the common available wild cards are shown in Table 1. Wild cards however do lose their meanings in some special circum-

Table 1: Wildcards recognized by the Shell

*	matches any number of characters including none
?	matches a single character
$[ijk]$	matches a single character in i, j or k
$[ijk]$	matches a single character not i, j or k
$[x-z]$	matches a single character in the specified range
$[!x-z]$	matches a single character not in the range x to z content...

stances. The * for example does not match a . at the start of a filename !!

We can also remove the special meaning associated with a wildcard manually by escaping the wildcard using the \ [backslash] operator.

0.2 Redirection

The shell leverages the concept of files to perform I/O. The file representing th standard input, standard output and standard error can all be used for redirection. The defaults for these files are the keyboard, monitor and the monitor. But all these standard files can be replaced during redirection.

We can redirect the standard output using the operators > and ». We can even redirect the combined outputs of multiple commands e.g.

```
(ls; who) > lsfile
```

Do remember that the shell does not differentiate between any kinds of files and *everything is a file.*

We can even redirect commands to take input through redirection. The command `wc` can count the number of lines , words and characters in a file. We can operate it in multiple ways e.g.

```
wc filename wc < filename
```

, where the <filename> is the filename to count words in. The < operator is used for the purpose of redirecting the standard input. Do you notice any difference in the two outputs ?

We can even combine the stand input and output redirection in the same command. If the standard input is required to be mixed with other inputs we use the symbol "-" to represent the standard input. E.g.

```
cat file1 - file2
```

We can redirect the standard error stream using the operator "2>". In fact the shell maintains a default numbering scheme for the different I/O streams given as follows:

0 - Standard input

1 - Standard output

2 - Standard error

Redirection of the standard error is shown in the example below:

```
cat filename 2> errorfile
```

As an interesting sidelight you can interchange any of the operators and filenames when combining streams.

0.3 Two special files

Two special files in the context of redirection are:

1. /dev/null - is used to incinerate any output that is of new use.
2. /dev/tty - send some output to your own terminal.

0.4 Piping commands

One of the special ability of the shell is to allow outputs of a command to be provided as input to the input of other commands. This is called *piping* and we use the | symbol to pipe commands in the shell.

We can also split the output from a command to different streams using the tee command. The command allows for the creation of a new stream while not changing the default stream of a command's output. As an example, observe the command:

```
wc | tee users.lst
```

You can even use tee multiple times in the same piping sequence.

0.5 Command Substitution

We can use the ‘ ‘ operator to embed a command inside another command in the shell. The shell first executes the nested command and then juxtapositions the output of the first command in the appropriate location in the second command. This feature is most useful when used in conjunction with the echo command as shown below:

```
echo The date today is ‘date‘
```

The bash shell allows the same functionality in a more readable form using the $() syntax

```
echo The date today is $(date)
```

0.6 Shell Variables

Shell variables are created using the assignment syntax as shown:

```
name=sachin
x=10
```

You can view the value of a variable by using the $ operator as shown:

```
echo $name
```

One interesting sidelight is the use of the single quote and the double quote w.r.t command substitution and variable evaluation. Both these operations can be escaped using the single quote but not using the double quotes. However, double quotes do escape other wildcards.

0.7 Assignments

For these exercises create five files in your home directory. These files should ba called "team1.txt", "team2.txt" ,"team3.txt", "team?.txt" and team123.txt.

1. Show the concatenated output from files team1.txt and team3.txt using wildcards.

2. Show the concatenated output from all files whose names start with team followed by a single character, which is then followed by ".txt".

3. Store the names of all files in your home directory that end with ".txt" in a file called atextfiles.txt.

4. Store the output of the date and calendar commands in a file called datetime.txt in a single command.

5. Send a message to the terminal used by the user abhijit.

6. Show the directory listing of a filename where the filename does not exist. What is a clean way of handling the error message.

7. Without providing any arguments to the wc command how would you count the words in a file and save the output in another file using a single command?

8. How many users are currently logged in the server?

9. We need to count the words in a file. The display of the count should appear on the screen as well as be saved in a file using a single command.

10. Produce the following line as an output using a single command : The number of words in the file team1.txt is <n>.

11. Show the line "The cost of the pair of shoes is $1000.

12. Write a script that shows the current directory, followed by today's date and the count of the users who are logged in.

13. Show the concatenated output from the files team1.txt and team2.txt. You should finish the display with a message entered by the user.

14. Enter a month and year from the user and show the calendar for that month.